



Input Snapshots Fusion for Scalable Discrete-Time Dynamic Graph Neural Networks

QingGuo Qi
qiqingguo@zju.edu.cn
Zhejiang University
Hangzhou, Zhejiang, China

Minhao Cheng
mmc7149@psu.edu
Pennsylvania State University
University Park, PA, USA

Hongyang Chen*
dr.h.chen@ieee.org
Zhejiang Lab
Hangzhou, Zhejiang, China

Han Liu
liu.han.dut@gmail.com
Dalian University of Technology
Dalian, Liaoning, China

Abstract

In recent years, there has been a surge in research on dynamic graph representation learning, primarily focusing on modeling the evolution of temporal-spatial patterns in real-world applications. However, within the domain of discrete-time dynamic graphs, the exploration of temporal edges remains underexplored. Existing approaches often rely on additional sequential models to capture dynamics, leading to high computational and memory costs, particularly for large-scale graphs. To address this limitation, we propose the Input Snapshots Fusion based Dynamic Graph Neural Network (SFDyG), which combines Hawkes processes with graph neural networks to capture temporal and structural patterns in dynamic graphs effectively. By fusing multiple snapshots into a single temporal graph, SFDyG decouples computational complexity from the number of snapshots, enabling efficient full-batch and mini-batch training. Experimental evaluations on eight diverse dynamic graph datasets for future link prediction tasks demonstrate that SFDyG consistently outperforms existing methods.

CCS Concepts

• **Theory of computation** → **Dynamic graph algorithms**; • **Information systems** → **Data mining**.

Keywords

Temporal Graph Embedding, Scalable Graph Neural Networks, Discrete-Time Dynamic Graph Representation, Hawkes Process

ACM Reference Format:

QingGuo Qi, Hongyang Chen, Minhao Cheng, and Han Liu. 2025. Input Snapshots Fusion for Scalable Discrete-Time Dynamic Graph Neural Networks. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709316>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, August 3–7, 2025, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1245-6/25/08
<https://doi.org/10.1145/3690624.3709316>

1 Introduction

Graphs, versatile data structures comprised of vertices and edges, play a pivotal role in various real-world applications such as social networks [8, 42], molecule graphs [31] and traffic networks [25, 45]. The rise of deep learning has elevated graph neural networks (GNNs) as essential tools for modeling graphs, enabling the depiction of intricate relationships and interactions among vertices via low-dimensional embeddings [10, 22, 38]. Currently, research in scalable graph representation learning has made significant advancements that can be applied to large-scale graphs with billions of nodes [3, 13, 41]. However, existing research primarily focuses on static graphs defined by fixed nodes and edges, thereby posing a challenge when transitioning to dynamic graphs.

Real-world graphs often undergo dynamic changes, with the graph structure evolving over time [30]. The basic changes may take place in both the quantity and attributes of nodes and edges in the temporal dimension. In this work, to streamline our investigation and align with existing literature [5, 30, 33, 34, 43, 44, 46, 48, 49], we take the assumption that nodes remain fixed and concentrate on the temporal edge events. Based on the granularity of time step [20, 47], dynamic graphs can be categorized into Continuous-time Dynamic Graphs (CTDGs) and Discrete-time Dynamic Graphs (DTDGs). In this case, CTDGs aim to predict upcoming events at the subsequent timestamp (or within a short period captured in a mini-batch), so they represent the evolution of dynamic graphs as a stream of edge events [7, 23, 33, 36, 39]. On the other hand, the goal of DTDGs is to forecast upcoming events within a defined time frame (e.g., a day, a week, or a month). This involves representing the dynamic graph through a chronological sequence of snapshots, where each snapshot encompasses all events occurring during that time interval. In this study, our primary focus lies on DTDGs for their ability to provide a comprehensive whole-graph perspective on dynamic graphs, which renders them more susceptible to scalability challenges.

The prevailing research methodology of DTDGs involves utilizing static GNN models [13, 38] to represent each graph snapshot separately. Subsequently, a sequential encoder [4, 16, 37] is employed to capture temporal dynamics for predicting all events in the subsequent snapshot [30, 34, 43, 44, 46, 48]. For example, as shown in Figure 1a, in order to forecast potential trades for the subsequent day using data from the previous week, cryptocurrency exchanges could utilize GNN on the daily snapshots to generate

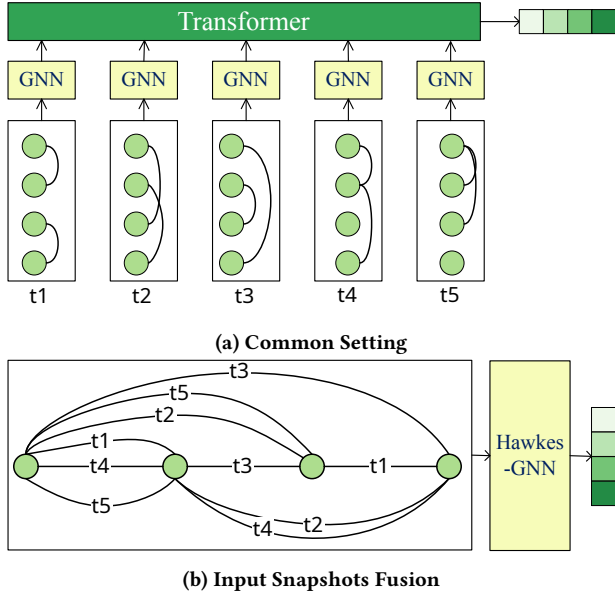


Figure 1: Training setting for the task of link prediction in dynamic graphs.

five embeddings for each user. These embeddings are then fed into a Transformer [37] to derive the user’s final embedding for prediction.

Simple as it is, this combination represents a compromise since *GNNs designed for static graphs are insufficient for handling multiple temporal edges*. Furthermore, applying a static GNN to process each snapshot comes with notable limitations. Precise timestamps are lost, resulting in equal treatment of all edges within a given time-frame, and multiple edges between two nodes are merged into one if they coexist. Moreover, incorporating sequential encoders increases system complexity as the number of input snapshots grows, which limits the model’s scalability on large datasets with long sequences, even when using mini-batch training methods such as GraphSAGE [13] or ClusterGCN [3]. Therefore, existing studies on the scalability of DGNNs, such as Roland [44] and Efficient-DGNN [1], often prioritize approaches that involve dividing the sequence. In this study, we take a reverse approach and propose a novel method termed “input snapshots fusion”, as shown in Figure 1b, which merges all input snapshots into a large temporal graph, enabling the presence of multiple temporal edges connecting two nodes.

In the temporal graph, every temporal edge is associated with a timestamp, denoting either a precise time point or a date, depending on the dataset. To represent temporal edges, we utilize the theory of Hawkes processes to capture events within the adjacency matrix. Interestingly, by induction, modeling on the temporal graph can be approximately viewed as a graph denoising problem under the smoothing assumption with time decay. In other words, the influence of an event relies on its age, with newer events carrying a stronger impact, as observed in applications like communication and recommendation systems. This formulation leads to the development of a message passing mechanism with time decay, which leads to the integration of Hawkes processes with Graph

Convolutional Networks [22] (Hawkes-GCN) and Graph Attention Networks [38] (Hawkes-GAT). These novel models allow us to effectively incorporate temporal information into GNN theory for modeling temporal graphs without loss of temporal edges, thereby enhancing expressiveness. Moreover, by utilizing a single temporal graph as input, both full-batch and mini-batch training methods are independent of the number of snapshots, enabling more effective scalability for large dynamic graphs.

Extensive experiments conducted on eight widely used public datasets demonstrate that our approach outperforms state-of-the-art baselines significantly in link prediction tasks. The adoption of the mini-batch training approach substantially reduces the demand for GPU memory, achieving a maximum reduction of 44% compared with full-batch training. Furthermore, ablation studies validate the effectiveness of the proposed Hawkes processes-based GNN in accurately modeling temporal graphs, resulting in a potential enhancement of up to 6 times compared to plain GAT.

In summary, the main contributions are as follows:

- We propose a novel idea of input snapshot fusion for discrete-time dynamic graphs, which merges multiple input snapshots into a single temporal graph structure. This approach facilitates a deeper understanding of the temporal evolution of dynamic graphs.
- We formulate the graph denoising problem using Hawkes processes theory under the assumption of time-decay smoothing in temporal graphs. This framework enables the design of efficient GNNs that incorporate a time-decay-based message-passing mechanism for dynamic graphs.
- We conduct extensive experiments to validate the effectiveness of our proposed model. The experiments demonstrate the superior performance of our approach in link prediction tasks on eight widely used public datasets.

2 Preliminaries

In this section, we introduce notations for DTDGs and briefly summarize several important models.

A discrete-time dynamic graph is defined as a series of snapshots $\{G^1, G^2, \dots, G^T\}$, where T is the total number of snapshots. The snapshot at time t , i.e., $G^t = (V^t, E^t)$ is a graph with a node set V^t and an edge set $E^t \subseteq V^t \times V^t$, where E_{ij}^t represents an edge from node i to node j at snapshot t . We use A^t to denote the binary adjacency matrix corresponding to the edge set E^t , and the neighbors of node i in G^t , denoted as $N(i)$, is a set of edges, $\{(i, j) \in E^t\}$. The superscript “ t ” can be omitted when it is not specified.

As shown in Figure 1b, a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T})$ was obtained by Input Snapshots Fusion which merges the snapshots. Specifically, the node set $\mathcal{V}$ is equal to V , while the edge set $\mathcal{E}$ represents the union of edges across the snapshots, with $\mathcal{T}$ are the timestamps of the snapshot associated with $\mathcal{E}$. It is important to note that, in practice, using the exact occurrence time of each edge as $\mathcal{T}$, when available, rather than the snapshot indices, may improve performance. However, this distinction does not affect our conclusion. To locate the temporal edges, the neighbors of a node i in $\mathcal{G}$, denoted as $N(i) = \{(i, j, \tau) \in (\mathcal{E}, \mathcal{T})\}$, form a set of edges

with timestamp that represent events involving nodes i and j at time τ .

In this paper, we denote the output of the l -th layer output of the GNN as $H^l \in \mathbb{R}^{n \times d}$, where $H^0 = X$ represents the input node features. We assume that the input node features remain fixed across different snapshots. Here $n = |V|$ denotes the number of nodes, d is the embedding dimension, and H_i refers to the i -th row of H . In the following, we outline several key models.

2.1 Link Prediction Tasks in DTDGs

A discrete-time dynamic graph consists of a sequence of snapshots, which are divided into training, validation, and test sets according to their sequential indices. As shown in Figure 1a, the model input is a sliding window of length w , where $w \geq 1$, a configurable hyper-parameter. The model employs spatial and temporal encoders to generate low-dimensional embeddings for each vertex. To predict edges in the subsequent snapshot, a multilayer perceptron (MLP) is used as a predictor, which takes the embeddings of two nodes as input and outputs the probability of a connection in the next time frame.

2.2 Graph Convolutional Networks

A single GCN layer [22] can be written as follows:

$$H^{l+1} = \hat{A}H^lW,$$

where $W \in \mathbb{R}^{d \times d}$ is a feature transformation matrix, and $\hat{A}$ is a normalized adjacency matrix. Additionally, from the perspective of message passing [10], the update formula for node i can be expressed as

$$H_i^{(l+1)} = \sum_{j \in N(i) \cup \{i\}} \frac{1}{\sqrt{\deg(i)} \cdot \sqrt{\deg(j)}} \cdot (W^\top H_j^{(l)}), \quad (1)$$

where the features of neighboring nodes are first transformed by the weight matrix W , normalized by degree, and then summed.

2.3 Graph Attention Networks

Graph Attention Networks (GAT) [38] adopts the same message passing mechanism as GCN. The feature aggregation operation for node i is defined as:

$$H_i^{l+1} = \sum_{j \in N(i)} \alpha_{ij} H_j^l, \quad \text{with} \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})}. \quad (2)$$

In this aggregation operation, α_{ij} represents the attention score that differentiates the importance of distinct nodes within the neighborhood. Specifically, α_{ij} is the normalized form of e_{ij} , which is defined as:

$$e_{ij} = \text{LeakyReLU} \left(\left[H_i' \| H_j' \right] a \right), \quad (3)$$

where $[\cdot \| \cdot]$ denotes the concatenation operation and $a \in \mathbb{R}^{2d}$ is a learnable vector.

2.4 GNNs as Solving Graph Denoising Problem

According to the conclusion in [28], GNN models can be regarded as an approximation for solving a graph denoising problem under the assumption of smoothness. Formally, Given a noisy input signal $S \in \mathbb{R}^{N \times d}$ on a graph G , the goals is to recover a clean signal

$F \in \mathbb{R}^{N \times d}$, assumed to be smooth over G , by solving the following optimization problem:

$$\arg \min_F \mathcal{L} = \|F - S\|_F^2 + \lambda \cdot \text{tr}(F^\top \mathbf{L} F). \quad (4)$$

Where the first term guides F to be close to S , while the second term $\text{tr}(F^\top \mathbf{L} F)$ is the Laplacian regularization which guides F 's smoothness over G , with $\lambda > 0$'s mediation.

2.5 Hawkes Process

The Hawkes process is a typical temporal point process [14] that describes a sequence of discrete events by assuming that previous events have an impact on the current, with the influence diminishing over time. A univariate Hawkes process is defined to be a self-exciting temporal point process whose conditional intensity function $\lambda(t)$ is defined to be

$$\lambda(t) = \mu(t) + \sum_{\tau_i < t'} \kappa(t' - \tau_i), \quad (5)$$

where $\mu(t)$ represents the background intensity, τ_i are the points in time occurring prior to time t' ; κ is an exciting function that models the time decay effect of history on the current event, which is usually in the form of an exponential function, $\kappa(t-s) = \exp(-\delta(t-s))$, where the δ is a non-negative source dependent parameter, representing the time sensitivity of the process.

With the advancement of graph representation learning, Hawkes processes have been incorporated into dynamic graph representation learning to model the influence of historical behaviors on current actions [49] and capture the dynamic nature of graph structures [27]. However, existing research primarily focuses on integrating Hawkes processes with GNNs within the local scope between node pairs in CTDG. In this paper, we will present how Hawkes processes are effectively integrated with GNNs from a global perspective within the adjacency matrix in the context of DTDG.

3 The SFDyG Model

In this section, we present the proposed SFDyG framework. First, we examine the feasibility and advantages of input snapshot fusion for DTDGs. Next, we explain why the Hawkes processes can enhance graph neural networks (GNNs) in handling temporal edges, demonstrating that modeling temporal graphs can be approximately viewed as a graph denoising problem with time-decayed edge weights. Furthermore, we illustrate how existing mini-batch training methods can be seamlessly applied to Hawkes-GNNs for link prediction tasks, enabling more effective scaling to large datasets.

3.1 Input Snapshots Fusion

As previously discussed, snapshots constitute the foundational structure of DTDGs. However, modeling snapshots separately is inefficient and introduces the risk of losing temporal information. Drawing from these insights, as illustrated in Figure 1b, we propose to fuse the input snapshots within the sliding window into a single temporal graph to improve the modeling of DTDG. Formally,

$$\mathcal{G} = \text{Fusion}(G^1, \dots, G^w) = (V, E^1 \cup \dots \cup E^w), \quad (6)$$

where $G^1, \dots, G^w$ are the input snapshots in the sliding window and $\cup$ represents union of two edge sets.

In the generated temporal graph, each temporal edge is associated with a time attribute, which may be an exact timestamp or a snapshot index, depending on the dataset. It is ensured that the sequence of events between any two nodes is ordered, and the temporal granularity across all events remains consistent. In this way, it is possible to more effectively utilize time information with long dependencies, as well as apply state-of-the-art algorithms and acceleration methods from static graph fields. However, the generated temporal graph $\mathcal{G}$ is beyond the capabilities of static GNNs, as multiple temporal edges can exist between two nodes.

3.2 Hawkes Processes Based GNN

Hawkes processes are powerful tools for modeling sequences of historical events, especially in scenarios where events recur, rendering them well-suited for modeling temporal edges. However, directly applying Hawkes processes to the temporal edges would result in $O(n^2)$ possible pair of nodes, making this approach infeasible to scale to large datasets.

To address this challenge, we propose to integrate Hawkes processes (Equation 5) with the message-passing framework (Equation 1) to update node embeddings rather than focus on node pairs. Specifically, the base rate $\mu(t)$ and self-excitation $\sum \kappa(t' - \tau)$ are captured by the linear layer and the message aggregation layer, respectively. Finally, to predict the probability of future links of node pairs, their embeddings are concatenated and processed through an MLP predictor.

Next, we connect Hawkes processes-based message passing neural networks with the graph denoising problem by introducing Hawkes excitation matrix C to gain a deeper understanding and derive formulations of Hawkes processes-based GNNs.

Definition 3.1 (Hawkes excitation matrix). a symmetric matrix C where each element C_{ij} represents the excitation effect of previously occurred events from node i to node j ,

$$C_{ij} = \sum_{(i,j,\tau) \in \mathcal{E}_{ij}} \exp(-\delta(t' - \tau)), \text{ with } \tau < t', \quad (7)$$

where t' denotes the starting point of the time frame to be predicted, while the non-negative scalar δ represents the time sensitivity parameter. It is noteworthy that the Hawkes excitation matrix shares the same shape and zero elements as the adjacency matrix, but their non-zero elements differ. In the Hawkes excitation matrix, non-zero elements are expressions that quantify the excitation effect of temporal edges rather than indicating the presence of an edge. Thus, the Hawkes excitation matrix can be regarded as an extension of the adjacency matrix in the context of temporal graphs.

Based on these definitions, the temporal graph denoising problem can be formulated by employing C in Equation 4, as described in Theorem 3.1.

THEOREM 3.1 (HAWKES TEMPORAL GRAPH DENOISING). *When we adopt the Laplacian matrix $\mathbf{L} = \mathbf{D} - \mathbf{C}$, where $\mathbf{D} = \text{diag}(\sum_j C_{ij})$, the graph denoising problem (Equation 4) was extended to temporal domain with the assumption that the smoothness of edges decays over time.*

PROOF.

$$\begin{aligned} \lambda \cdot \text{tr}(\mathbf{F}^\top \mathbf{L} \mathbf{F}) &= \lambda \cdot \sum_{(i,j) \in E} C_{ij} \|\mathbf{F}_i - \mathbf{F}_j\|_2^2 \\ &= \lambda \cdot \sum_{(i,j) \in E} \sum_{(i,j,\tau) \in \mathcal{E}_{ij}} \exp(-\delta(t' - \tau)) \|\mathbf{F}_i - \mathbf{F}_j\|_2^2 \\ &= \lambda \cdot \sum_{(i,j,\tau) \in \mathcal{E}} \exp(-\delta(t' - \tau)) \|\mathbf{F}_i - \mathbf{F}_j\|_2^2. \end{aligned} \quad (8)$$

Where each element of the summation is in the form of a time decay coefficient multiplied by a graph smoothing indicator. Which completes the proof. $\square$

Theorem 3.1 demonstrates that, in Hawkes processes-based temporal graph denoising problem, the influence of a temporal edge on its two endpoints depends on both the age of the edge and the time sensitivity parameter δ . In other words, the Hawkes excitation matrix enables the graph denoising method to handle temporal graphs under the assumption that the smoothness constraints imposed by temporal edges diminish over time.

Then, Hawkes processes based GNNs can be derived by applying the findings from [28], which demonstrate that the graph denoising problem can be approximately regarded as a GNN model.

3.2.1 Hawkes-GCN. By adopting a normalized Hawkes Laplacian matrix, defined as $\mathbf{L} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{D} - \mathbf{C})\mathbf{D}^{-\frac{1}{2}}$ for Equation 4, where $\mathbf{D} = \text{diag}(\sum_j A_{ij})$ is the diagonal degree matrix derived from the binary adjacency matrix, the temporal graph denoising problem is connected to the Hawkes-GCN model [28]. Formally,

$$\mathbf{H}^{k+1} = \mathbf{D}^{-\frac{1}{2}} \mathbf{C} \mathbf{D}^{-\frac{1}{2}} \mathbf{H}^k \mathbf{W}, \quad (9)$$

specifically, the i -th element in the node embedding H can be expressed as follows,

$$\begin{aligned} H_i^{(k+1)} &= \sum_{j \in N(i)} \frac{C_{ij}}{\sqrt{\deg(i)\deg(j)}} \cdot \mathbf{W}^\top H_j^{(k)} \\ &= \sum_{(i,j,\tau) \in N(i)} \frac{\exp(-\delta_i(t' - \tau))}{\sqrt{\deg(i)\deg(j)}} \mathbf{W}^\top H_j^{(k)}. \end{aligned} \quad (10)$$

Where δ_i is a learnable parameter indicating that the time sensitivity parameter depends on the source node i . Through Equation 10, it can be observed that, during the message aggregation process in the temporal graph, each temporal edge transmits messages whose influence diminishes over time. Accordingly, we denote this Hawkes processes-based GNN as a time-decayed message-passing neural network. In addition, we use the diagonal matrix derived from the binary adjacency matrix for regularization to incorporate the information of the unique number of neighbors for each node, which generally results in better performance. However, for dense temporal graphs, we recommend applying batch normalization [18] to mitigate potential numerical instabilities.

3.2.2 Hawkes-GAT. The success of GAT is to calculate the non-negative attention score α_{ij} to differentiate the importance of distinct nodes in the neighborhood, which is a natural choice of the time sensitivity coefficient δ in Equation 10. Then, the GAT model

is extended to the temporal domain, Formally,

$$H_i^{(k+1)} = \sum_{(i,j,\tau) \in N(i)} \frac{\exp(-\delta_{ij}(t' - \tau))}{\sqrt{\deg(i)\deg(j)}} W^T H_j^{(k)}, \quad (11)$$

$$\text{with } \delta_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})}.$$

Where e_{ij} is described in Equation 3. In this way, the Hawkes-GAT is derived under the assumption that the time sensitivity depends on both the source and target node of an edge, enabling more flexible modeling.

3.3 Loss Function

The link prediction task for the discrete-time dynamic graphs involves two steps [30, 34, 44]: first, generating all node embeddings H using an encoder, denoted as GNN. Second, predicting the target link probability based on the embeddings of its two endpoints, using an MLP referred to as Fc. The forward propagation process can be formalized as:

$$H = \text{GNN}(G_{t-w}, \dots, G_{t-1}), \hat{y}_{ij} = \sigma(\text{Fc}(H_i, H_j)). \quad (12)$$

where H_i represents the embedding of node i , and $\hat{y}_{ij}$ denotes the likelihood of a future connection from node i to node j . The benefit of this two steps model is that node embeddings can be reused, which is especially suitable for training with multiple negative samples. Consistent with existing research, the cross-entropy loss is used as the loss function:

$$\mathcal{L} = \frac{1}{|E|} \sum_{e_{ij} \in E} (-y_{ij} \cdot \log(\hat{y}_{ij}) - (1 - y_{ij}) \cdot \log(1 - \hat{y}_{ij})). \quad (13)$$

Where the term E represents the collection of all positive edges union all randomly sampled negative edges.

3.4 Minibatch Training Algorithm

In this subsection, we demonstrate that the existing mini-batch algorithm can be efficiently applied to our proposed Hawkes processes based GNNs for link prediction tasks.

While training efficiency is improved by decoupling from window length through Input Snapshots Fusion, negative sampling plays a critical role in redundant computations. To further enhance efficiency, we adopt single negative sampling, as demonstrated in our experiments in section 4, which confirms its sufficiency. Additionally, the link neighbor loader in PyG [9] is employed for mini-batching temporal graphs, enabling a straightforward mini-batch algorithm, as detailed in Algorithm 1.

Compared to other models, mini-batching with Hawkes-GNN is more memory efficient as it stores the node embeddings h only once. Additionally, it runs faster as there is no additional temporal encoder, which makes it more scalable to large datasets. It is important to emphasize that Algorithm 1 differs significantly from the mini-batch training approach used in CTDG [33]. Specifically, it processes the dynamic graph from a whole graph perspective, encompassing nodes over a large time frame, rather than focusing on a subgraph within a small time interval. Moreover, the steps can be executed in parallel rather than sequentially in chronological order, making it more efficient with parallel computation.

Algorithm 1: One Epoch of Mini-batch Training for SFDyG

Input: Encoder GNN, decoder Fc, window length w and the dynamic graph $\{G^1, G^2, \dots, G^T\}$

Output: Updated GNN, Fc

```

1 for index  $i$  from  $w+1$  to  $T$  do
2    $\mathcal{G} \leftarrow \text{Fusion}(G^{i-w}, \dots, G^{i-1})$ ;
3    $\text{loader} \leftarrow \text{LinkNeighborLoader}(\mathcal{G}, G^i.\text{Edge})$ ;
4   for batch in loader do
5      $h \leftarrow \text{GNN}(\text{batch})$ ;
6      $\hat{y} \leftarrow \text{Fc}(h, \text{batch.edge\_label\_index})$ ;
7      $y \leftarrow \text{batch.edge\_label}$ ;
8      $\text{loss} \leftarrow \text{cross-entropy}(\hat{y}, y)$ ;
9      $\text{loss.backward}()$ ;
10     $\text{optimizer.step}()$ ;
11  end
12 end
```

Table 1: Comparison of time and memory complexities.

Method	Time	Memory
DySAT	$O(tlef + tlnf^2)$	$O(tlnf + tlf^2)$
EvolveGCN	$O(tlef + tlnf^2)$	$O(tlnf + tlf^2)$
Roland	$O(lef + lnf^2)$	$O(lef + lf^2)$
SFDyG-F	$O(tlef + lnf^2)$	$O(tlef + lf^2)$
SFDyG-M	$O(2ted^l f^2)$	$O(2bd^l f + lf^2)$

3.5 Complexity Analysis

In this subsection, time and space complexity analyses were provided for SFDyG and the following representative DGNNs: DySAT [34], EvolveGCN [30] and ROLAND [44].

In the context of discrete-time dynamic graph link prediction, we consider several parameters to analyze the complexity of the algorithms. The snapshots are arranged in a slice window with length t , and the total number of nodes is represented as $n = |V|$. Moreover, we have the average number of edges per snapshot denoted as $e = |E|$, and the average degree per node as d . Without loss of generality, we assume that the node feature dimension and the length of the hidden vectors in the network are both f . Additionally, the number of layers in the graph neural network is represented as l , and the batch size for mini-batch training is denoted as b . The time and memory complexities are summarized in Table 1, while detailed analyses are provided in Appendix A.2.

Overall, the full-batch version denoted as SFDyG-F exhibits the second-best time and space complexity, only surpassed by Roland. In comparison to Roland, SFDyG demonstrates enhanced capability in capturing long-range temporal dependencies, without being constrained by sequential training in chronological order. Typically we have $l \leq 3$ for GNN and large graphs tend to exhibit sparsity, therefore, the mini-batch version denoted as SFDyG-M showcases superior space complexity, making it suitable for application in large-scale dynamic graphs with numerous nodes.

Table 2: Overall performance (MRR@100) comparison on eight datasets (% is omitted). Our experiments guarantee consistent data settings and standardized methods for computing Mean Reciprocal Ranks (MRRs) to facilitate fair comparisons. Each experiment is conducted using three random seeds, and the average performance is reported along with the standard error.

Methods	OTC	Alpha	UCI	Title	Body	AS733	SBM	SO
DySAT	21.39 ± 2.79	19.16 ± 2.21	23.31 ± 9.42	17.46 ± 4.18	13.87 ± 3.90	25.10 ± 1.71	6.88 ± 0.53	OOM.
EvolveGCN	7.84 ± 0.09	6.65 ± 0.55	7.33 ± 0.15	30.67 ± 0.00	18.55 ± 0.02	42.06 ± 0.00	21.38 ± 0.00	31.21 ± 0.48
Roland	30.94 ± 0.70	32.97 ± 1.78	17.04 ± 2.30	46.33 ± 0.27	<u>38.57 ± 0.42</u>	21.21 ± 5.73	1.96 ± 0.00	38.57 ± 1.44
WinGNN	3.86 ± 1.26	3.90 ± 0.84	2.37 ± 0.13	4.19 ± 1.25	2.69 ± 0.38	4.29 ± 2.10	3.35 ± 0.50	7.51 ± 0.67
VGRNN	6.62 ± 0.10	6.49 ± 0.29	6.96 ± 0.08	OOM.	17.19 ± 0.14	41.94 ± 2.04	19.79 ± 0.23	OOM.
HTGN	6.36 ± 0.06	7.72 ± 0.66	8.67 ± 0.43	11.50 ± 0.98	10.70 ± 0.52	13.86 ± 0.58	10.92 ± 1.19	OOM.
GraphMixer	43.67 ± 0.25	35.72 ± 0.41	33.63 ± 0.02	38.32 ± 0.01	33.15 ± 0.02	28.86 ± 0.00	1.96 ± 0.00	OOM.
M2DNE	7.82 ± 1.05	5.49 ± 0.29	8.86 ± 0.44	5.40 ± 0.05	6.03 ± 0.38	19.43 ± 0.12	OOM.	OOM.
GHP	3.40 ± 0.41	3.40 ± 0.46	4.15 ± 0.14	16.00 ± 2.32	8.33 ± 2.00	22.15 ± 4.88	26.85 ± 17.65	OOM.
Hawkes-GCN	<u>46.16 ± 0.45</u>	47.87 ± 5.85	35.61 ± 0.06	<u>47.44 ± 0.20</u>	36.44 ± 0.42	<u>44.34 ± 0.41</u>	29.10 ± 0.73	<u>46.41 ± 0.31</u>
Hawkes-GAT	51.34 ± 0.07	<u>40.66 ± 0.25</u>	<u>35.59 ± 1.58</u>	50.84 ± 0.05	40.97 ± 0.47	45.95 ± 0.79	<u>28.96 ± 0.70</u>	48.83 ± 0.14

Table 3: Summary of dataset statistics.

	# Nodes	# Edges	# Time Steps (Train/Val/Test)	Avg. Degree
UCI	1,899	59,835	35 / 5 / 10	0.36
Alpha	3,777	24,173	95 / 13 / 28	0.04
OTC	5,881	35,588	95 / 14 / 28	0.05
Title	54,075	571,927	122 / 35 / 17	0.06
Body	35,776	286,562	122 / 35 / 17	0.05
AS733	7,716	1,167,892	70 / 10 / 20	2.12
SBM	1,000	4,870,863	35 / 5 / 10	97.42
SO	2,601,997	63,497,050	65 / 9 / 18	0.12

4 Experiments

4.1 Experimental Setup

4.1.1 Datasets. We conducted experiments on eight commonly used public datasets that have been extensively evaluated in previous studies on dynamic graph representation learning, encompassing Bitcoin-Alpha, Bitcoin-OTC, UCI, Reddit-Title, Reddit-Body, AS733, and Stack Overflow. The fundamental statistics of the eight datasets are presented in Table 3. Following EvolveGCN [30], we subdivided the original dataset into multiple snapshots of equal frequency. Subsequently, the training, validation, and test sets are divided along the time dimension. Details of the datasets can be found in Appendix B.1.

4.1.2 Baselines. We evaluated the performance of our proposed model, Hawkes-GCN, Hawkes-GAT by comparing to several dynamic GNN baselines, namely EvolveGCN [30], DySAT [34], GHP [35], ROLAND [44], GraphMixer (G-Mixer for short) [6], M2DNE [27], VGRNN [12], HTGN [43] and WinGNN [48]. Note that some baseline models were originally designed for modeling the dynamics of CTDG. To demonstrate the superiority of Hawkes-GNN, we reimplemented these models and adapted them to the DTDG setting. In Appendix B.2, a comprehensive description of these baselines can be found.

4.1.3 Evaluation metrics. We evaluate the effectiveness of the SFDyG framework in the context of future link prediction. Our primary evaluation metric is the Mean Reciprocal Rank (MRR) with 100 negative sampling as defined in OGB [17], which is an average of the pessimistic and optimistic ranks. An analysis of different MRRs based on various statistical approaches is presented in Appendix B.3.

4.1.4 SFDyG Architecture. SFDyG adopts the prevalent encoder-decoder architecture for future link prediction, featuring a two-layer Hawkes processes based GNN as the encoder to generate embeddings for all nodes. The model utilizes a two-layer MLP as the decoder, taking a pair of nodes as input and determining the probability of their forthcoming connection. To maintain parity in evaluations, all models in the experiment share identical decoders architecture, differing only in their encoders.

4.2 Main Results

4.2.1 Full-batch Training. The performance of the proposed SFDyG and other baseline models in dynamic link prediction is presented in Table 2. The results reveal significant variations in the effectiveness of existing baseline models across different dynamic graph datasets. DySAT and EvolveGCN, equipped with temporal encoders, demonstrate better performance on denser graphs like AS733 and SBM, suggesting possible underutilization of temporal encoders. Conversely, models with single snapshot inputs, such as VGRNN and HTGN, exhibit similar performances, indicating potential neglect of temporal features. GraphMixer and Roland emerge as the top among the baselines, however, they fail to learn on dense datasets like SBM due to limited history neighbors and completed temporal patterns. Hawkes processes based methods like M2DNE and GHP behave better on dense datasets, but they are likely to OOM and behave poorly on sparse datasets like OTC and Alpha. In contrast, our proposed model SFDyG showcases substantial advantages over all datasets, outperforming baseline models by a considerable margin, highlighting the efficacy of our GNN based on Hawkes processes in capturing temporal information in temporal graphs. Generally,

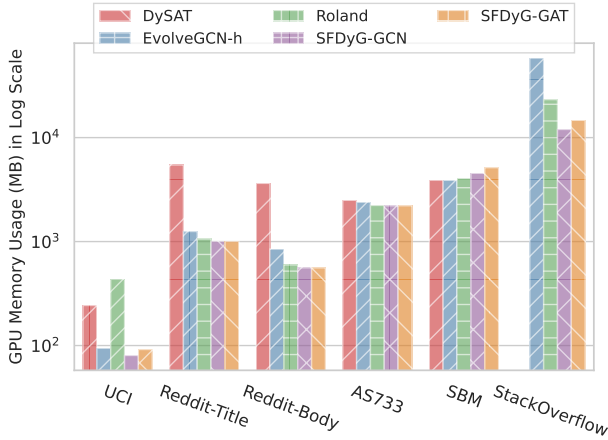


Figure 2: The GPU memory usage of SFDyG and representative baselines on six datasets.

Table 4: The performance of full-batch training and mini-batch training was compared on eight datasets in terms of the relative percentage change in MRR@100 (Δ MRR@100) and GPU memory usage (Δ GPU).

Dataset	Full-batch	Mini-batch	Δ Mrr@100	Δ GPU
OTC	51.34 $\pm$ 0.07	52.59 $\pm$ 1.49	$\uparrow$ 2.43%	$\downarrow$ 4.13%
Alpha	40.66 $\pm$ 0.25	40.61 $\pm$ 0.19	$\downarrow$ 0.12%	$\downarrow$ 13.01%
UCI	35.59 $\pm$ 1.58	39.00 $\pm$ 0.11	$\uparrow$ 9.58%	$\downarrow$ 0.67%
Title	50.84 $\pm$ 0.05	51.17 $\pm$ 0.17	$\uparrow$ 0.65%	$\downarrow$ 6.59%
Body	40.97 $\pm$ 0.47	41.87 $\pm$ 0.18	$\uparrow$ 2.20%	$\downarrow$ 7.78%
AS733	45.95 $\pm$ 0.79	52.59 $\pm$ 1.18	$\uparrow$ 14.45%	$\downarrow$ 2.03%
SO	48.83 $\pm$ 0.14	47.84 $\pm$ 0.05	$\downarrow$ 2.03%	$\downarrow$ 44.01%
SBM	28.96 $\pm$ 0.70	29.45 $\pm$ 0.54	$\uparrow$ 1.69%	$\uparrow$ 0.19%

Hawkes-GAT demonstrates slightly superior performance compared to Hawkes-GCN, except for datasets with minimal edges like bitcoin-alpha and nodes like SBM, thereby affirming the more adaptable modeling capability of Hawkes-GAT. Collectively considering these factors, we assert that SFDyG surpasses existing baseline methods in dynamic link prediction capabilities.

Figure 2 illustrates the utilization of GPU memory by the SFDyG model throughout the training procedure as compared to various standard baseline methods. The findings demonstrate a notable superiority of our approach over other multi-snapshot input baselines, some of which face out-of-memory (OOM) issues when applied to the StackOverflow dataset. GPU memory consumption of our proposed methods closely resembles that of the single snapshot input method Roland, exhibiting a lower constant space complexity, albeit performing less effectively than Roland when applied to the SBM dataset with an edge degree of about 100. The higher efficiency in memory usage of our method suggests promising scalability potential.

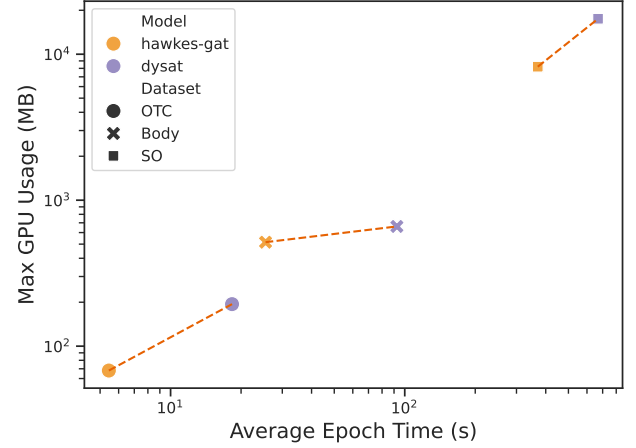


Figure 3: The comparison of average epoch time and maximum GPU memory usage during mini-batch training. The dotted line represents one dataset, while the endpoints illustrate the execution metrics on that dataset.

4.2.2 Mini-batch Training vs. Full batch Training. We train our proposed Hawkes-GAT using the mini-batch training algorithm on eight datasets to compare the memory consumption and performance between full-batch and mini-batch training. The batch size is determined according to the size of the dataset, and all neighbors are selected by the neighbor sampler.

The comparative results of full-batch and mini-batch training are presented in Table 4. In general, there is a noticeable decrease in GPU consumption in the majority of cases, particularly with large datasets such as StackOverflow, showing a reduction of more than 44%. However, for smaller and denser datasets like SBM, there is a slight increase of 0.19% in GPU consumption, suggesting that mini-batch training approaches may not be ideal in these particular situations. Moreover, the experimental results exhibit minimal variance in most datasets except on UCI and AS733 increased by 9.58% and 14.45% respectively. This phenomenon may be attributed to the capacity of small-batch training to mitigate the influence of supernodes.

4.2.3 Mini-batch Training vs. Mini-batch Training. In the existing literature, few studies have explored the application of mini-batch training to DTDGs. A key challenge is aligning selected nodes and their neighbors across snapshots. Unexpectedly, we find that Input Snapshots Fusion helps address this issue. Specifically, for each snapshot, an additional attribute is added to every edge to record the snapshot’s index. The mini-batch method is then applied to the fused temporal graph to generate temporal subgraphs, ensuring that all historical neighbors of the selected nodes are sampled. Finally, based on the snapshot index stored in the edge attributes, the temporal subgraph is divided back into multiple sub-snapshots. This approach enables traditional multi-snapshot models to leverage mini-batch training, thereby scaling effectively to large datasets.

Based on the above analysis, we evaluated mini-batch training on DySAT and Hawkes-GAT by measuring the average single-epoch

Table 5: The overall performance comparison (MRR@100) between the plain GAT and the Hawkes-GAT, with the relative percentage of improvements.

DataSet	GAT	Hawkes-GAT	Improve
OTC	15.18 $\pm$ 7.63	51.34 $\pm$ 0.07	238.21%
Alpha	9.68 $\pm$ 3.17	40.66 $\pm$ 0.25	320.04%
UCI	15.98 $\pm$ 4.06	35.59 $\pm$ 1.58	122.72%
Title	17.87 $\pm$ 2.40	50.84 $\pm$ 0.05	184.50%
Body	12.33 $\pm$ 2.22	40.97 $\pm$ 0.47	232.28%
AS733	22.62 $\pm$ 1.04	45.95 $\pm$ 0.79	103.14%
SBM	3.78 $\pm$ 1.87	28.96 $\pm$ 0.70	666.14%
SO	22.39 $\pm$ 2.48	48.83 $\pm$ 0.14	118.09%

training time and maximum GPU memory usage across datasets of varying sizes, as shown in Figure 3. The three datasets are UCI, Body, and SO, representing small, medium, and large datasets, respectively. The results indicate that, under the same batch size, Hawkes-GAT consistently achieves better scalability with faster training speed and lower memory usage.

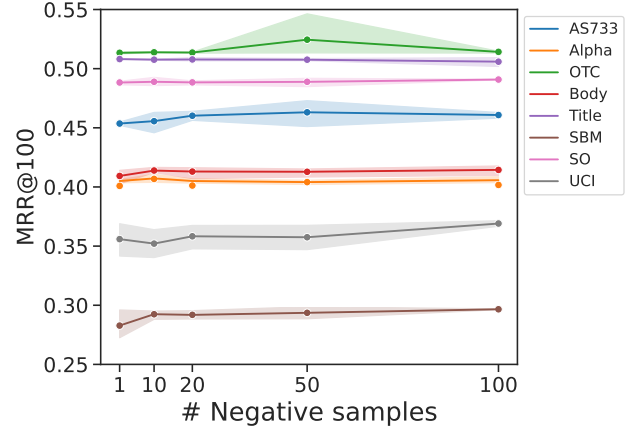
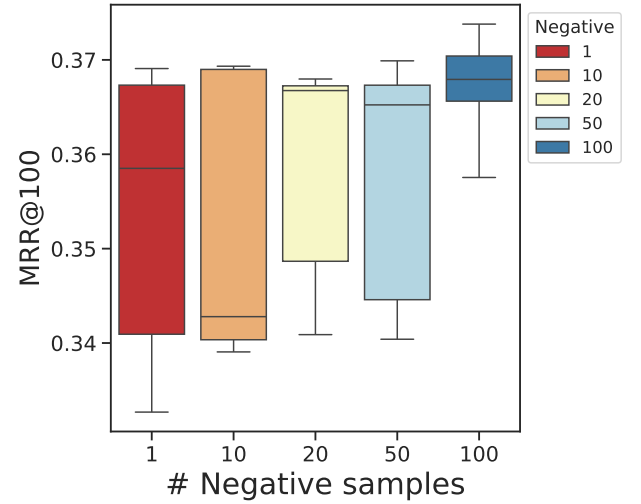
4.3 Ablation Study

In modern graph neural network libraries such as PyG [9], the data structures for static graphs and temporal graphs are identical. This implies that plain graph neural network algorithms, like GAT, can be utilized on temporal graphs formed through input snapshots fusion methods. Hence, the question arises: Is it necessary to develop dedicated algorithms for temporal graphs? To answer this question, we run experiments between the plain GAT and Hawkes-GAT.

As presented in Table 5, the findings reveal that the Hawkes processes-based GAT model significantly enhances the overall performance, surpassing the standard GAT by a considerable margin across all datasets. Particularly, on the SBM dataset, the Mrr@100 metric exhibited a notable increase, skyrocketing from 3.78% to 28.96%. This notable improvement underscores the efficacy and versatility of our proposed methodology.

4.4 Hyper-parameter Sensitivity Analysis

4.4.1 Negative sampling. helps train the model to distinguish between positive and negative pairs, which is widely used in link prediction tasks. Let k denote the number of negative samples for training. Previous works tend to have k larger than one. While k is essential for the efficiency of the mini-batch algorithm, we study the model performance with varying numbers of negative samples. As shown in Figure 4, the average model performance across the eight datasets does not exhibit significant changes as k increases. Take the UCI dataset as an example (Figure 5), an analysis using ANOVA [11] on the five experiment groups resulted in a p-value of 0.45969, which fails to reject the null hypothesis, suggesting no substantial variance in the means across these experimental sets. Consequently, we have the conclusion that the average performance of the proposed method is insensitive to k for our proposed method.

**Figure 4: Performance of Hawkes-GAT with varying numbers of negative samples during training.****Figure 5: The distribution of MRR@100 on the UCI dataset with different negative samples for training.**

4.4.2 Sliding window size. serves as another key hyperparameter. To investigate the model's sensitivity to the window size, various window sizes were used in training the Reddit-Title dataset to analyze the performance variations, as illustrated in Figure 6. The results demonstrate that a window size of 4 represents a critical point in the model's performance, indicating the presence of long-distance temporal dependencies within the data. Insufficient input snapshots lead to the omission of essential information. Furthermore, depicted in the figure, beyond a certain window size, the impact of the number of input windows on future predictions notably diminishes. This observation aligns with the Hawkes process assumption used in this study, suggesting that the influence of past

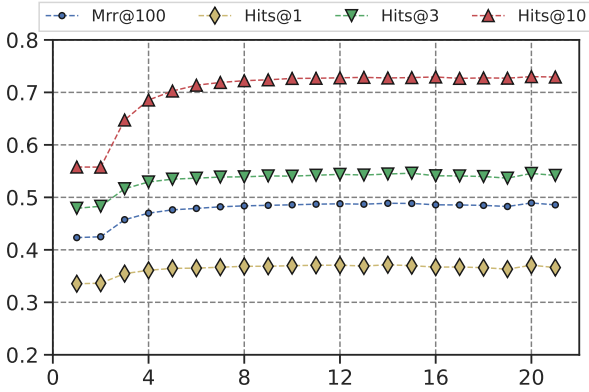


Figure 6: Performance of Hawkes-GAT on Reddit-Title with various sliding window sizes.

events on future predictions gradually wanes over time. Taken together, these findings underscore the robustness of our proposed Hawkes-GNNs.

5 Related Works

5.1 Dynamic Graph Neural Networks

Dynamic graph representation learning aims to learn the temporal low-dimensional representations of nodes as the graph evolves over time, mainly categorized into continuous-time dynamic graph (CTDG) and discrete-time dynamic graph (DTDG) methods based on the form of dynamic graphs. CTDG based methods treat dynamic graphs as event streams with accurate timestamps, generating dynamic node embeddings by iteratively processing information gathered from temporal neighbors [23, 36]. Despite the success of continuous-time methods [7, 33, 39], in practical applications, many datasets lack precise timestamp information due to historical reasons, making continuous-time methods inapplicable. In addition, while our work adopts the concept of temporal graphs from CTDG, it differs in its approach under the DTDG framework by modeling and predicting the entire graph as a whole. This distinction sets our research apart from prior studies.

DTDG represents the dynamic graph through a sequence of snapshots. It leverages multiple previous snapshots to predict the events in subsequent snapshots. The intuitive idea is to combine GNN with a time encoder such as RNN [4, 16] or Transformer [37], which, however, has high time complexity and space complexity. Recent studies have highlighted that the incorporation of extra time encoders can lead to overfitting [48]. Alternatively, a single snapshot as input has been explored by employing latent random variables [12], hyperbolic space [43] or meta-learning [44]. These approaches enable the system to process a single snapshot as input, offering a promising solution for scalability. By sequentially training the snapshots, existing static graph scaling techniques can be utilized. However, it should be noted that the training time scales linearly with the total number of snapshots. Moreover, disregarding long-distance time dependencies may lead to overfitting issues. What sets our research apart from existing methodologies is that we fuse

the multiple input snapshots, thereby obviating the necessity for additional temporal encoders.

5.2 Scalable Graph Neural Networks

GNNs are typically executed in a full-batch manner, which makes it challenging to scale to large graphs in practice for limited GPU memory and training time. To facilitate training on large-scale datasets, mini-batch training methods were proposed for static GNNs such as graph sampling [2, 3, 13], graph sparsification [24, 32], and graph partitioning [3, 29]. The basic idea is training the network on only a subset of the entire graph data at a time, rather than using the entire graph. The same issue also arises in DTDGs with the additional challenge that there are multiple snapshots. Few studies have explored how to align selected nodes across snapshots for mini-batch training on DTDGs. In contrast, our proposed Input Snapshots Fusion facilitates mini-batch training on both multiple snapshots and temporal graph approaches, enhancing the scalability of large datasets.

5.3 Hawkes Processes Based Graph Learning

Hawkes processes are powerful temporal point processes widely used in modeling event sequences [15], and have been extensively studied to adapt to different scenarios. Prior studies [19, 27, 35, 40, 50] often adhere to the formulation of the Equation 5 to predict the probability of future links, where the neighbor influence component duplicates message aggregation mechanism in GNNs. In contrast, we propose a deeper integration of Hawkes processes with GNNs through time-decayed message passing which eliminates the need for additional parameters from Hawkes processes, setting our approach apart from previous works.

6 Conclusion

This study proposes a novel approach to enhance the scalability of discrete-time dynamic graph models by combining multiple snapshots within the input sliding window into a single temporal graph. This method effectively decouples computational complexity from the number of snapshots, enabling the use of mini-batch training methods. To model the generated large temporal graph, we employ Hawkes excitation matrix to represent the temporal edges, which provides modeling of the temporal graph as denoising with time decay smoothing assumption. Building on this, we propose Hawkes processes-based GNNs, which capture graph dynamics effectively while being more resource-efficient than previous methods. Extensive experiments demonstrate the scalability, robustness, and versatility of our framework. This research focuses solely on the link prediction task in discrete-time dynamic graphs due to data constraints. Future directions involve exploring scalable methods such as graph partition and expanding our framework to diverse dynamic graph representation learning tasks including node classification and link classification.

Acknowledgments

Q. Qi and H. Chen were supported in part by the National Key Research and Development Program of China(2023YFB4502400) and the National Natural Science Foundation of China under Grant 62271452.

References

- [1] Venkatesan T. Chakaravathy, Shivmaran S. Pandian, Saurabh Raj, Yogish Sabharwal, Toyotaro Suzumura, and Shashanka Ubaru. 2021. Efficient scaling of dynamic graph neural networks. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) (SC '21). Association for Computing Machinery, New York, NY, USA, Article 77, 15 pages. <https://doi.org/10.1145/3458817.3480858>
- [2] Jie Chen, Tengfei Ma, and Cao Xiao. 2018. FastGCN: Fast learning with graph convolutional networks via importance sampling. In *International Conference on Learning Representations*. International Conference on Learning Representations, ICLR.
- [3] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 257–266.
- [4] Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *NIPS 2014 Workshop on Deep Learning, December 2014*.
- [5] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do We Really Need Complicated Model Architectures For Temporal Networks?. In *The Eleventh International Conference on Learning Representations*.
- [6] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do We Really Need Complicated Model Architectures For Temporal Networks?. In *The Eleventh International Conference on Learning Representations*.
- [7] da Xu, chuanwei ruan, evren korpceoglu, sushant kumar, and kannan achan. 2020. Inductive representation learning on temporal graphs. In *International Conference on Learning Representations (ICLR)*.
- [8] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *The World Wide Web Conference*. <https://doi.org/10.1145/3308558.3313488>
- [9] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [10] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. PMLR, 1263–1272.
- [11] Ellen R Girden. 1992. ANOVA: Repeated measures. Number 84. Sage.
- [12] Ehsan Hajiramezanali, Arman Hasanzadeh, Krishna Narayanan, Nick Duffield, Mingyuan Zhou, and Xiaoning Qian. 2019. Variational graph recurrent neural networks. *Advances in neural information processing systems* 32 (2019).
- [13] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [14] Alan G. Hawkes. 1971. Spectra of some self-exciting and mutually exciting point processes. *Biometrika* 58 (1971), 83–90. <https://api.semanticscholar.org/CorpusID:14122089>
- [15] Alan G Hawkes. 1971. Spectra of some self-exciting and mutually exciting point processes. *Biometrika* 58, 1 (1971), 83–90.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [17] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [18] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*. pmlr, 448–456.
- [19] Yugang Ji, Tianrui Jia, Yuan Fang, and Chuan Shi. 2021. Dynamic Heterogeneous Graph Embedding via Heterogeneous Hawkes Process. In *Machine Learning and Knowledge Discovery in Databases. Research Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part I* (Bilbao, Spain). Springer-Verlag, Berlin, Heidelberg, 388–403. https://doi.org/10.1007/978-3-030-86486-6_24
- [20] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. 2020. Representation learning for dynamic graphs: A survey. *The Journal of Machine Learning Research* 21, 1 (2020), 2648–2720.
- [21] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [22] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.
- [23] Srikanth Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 1269–1278.
- [24] Jiayu Li, Tianyun Zhang, Hao Tian, Shengmin Jin, Makan Fardad, and Reza Zafarani. 2020. SgcN: A graph sparsifier based on graph convolutional networks. In *Advances in Knowledge Discovery and Data Mining: 24th Pacific-Asia Conference, PAKDD 2020, Singapore, May 11–14, 2020, Proceedings, Part I 24*. Springer, 275–287.
- [25] Mengzhang Li and Zhanxing Zhu. 2021. Spatial-temporal fusion graph neural networks for traffic flow forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35, 4189–4196.
- [26] Ilya Loshchilov and Frank Hutter. 2016. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983* (2016).
- [27] Yuanfu Lu, Xiao Wang, Chuan Shi, Philip S. Yu, and Yanfang Ye. 2019. Temporal Network Embedding with Micro- and Macro-dynamics. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) (CIKM '19). Association for Computing Machinery, New York, NY, USA, 469–478. <https://doi.org/10.1145/3357384.3357943>
- [28] Yao Ma, Xiaorui Liu, Tong Zhao, Yozen Liu, Jiliang Tang, and Neil Shah. 2021. A unified view on graph neural networks as graph signal denoising. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 1202–1211.
- [29] Vasimuddin Md, Sanchit Misra, Guixiang Ma, Ramanarayan Mohanty, Evangelos Georganas, Alexander Heinecke, Dhiraj Kalamkar, Nesreen K Ahmed, and Sasikanth Avancha. 2021. Distgcn: Scalable distributed training for large-scale graph neural networks. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–14.
- [30] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*.
- [31] Patrick Reiser, Marlen Neubert, André Eberhard, Luca Torresi, Chen Zhou, Chen Shao, Houssam Metni, Clint van Hoese, Henrik Schopmans, Timo Sommer, et al. 2022. Graph neural networks for materials science and chemistry. *Communications Materials* 3, 1 (2022), 93.
- [32] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2019. DropEdge: Towards Deep Graph Convolutional Networks on Node Classification. In *International Conference on Learning Representations*.
- [33] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML 2020 Workshop on Graph Representation Learning*.
- [34] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. 2020. Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th international conference on web search and data mining*, 519–527.
- [35] Jin Shang and Mingxuan Sun. 2019. Geometric Hawkes Processes with Graph Convolutional Recurrent Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 01 (Jul. 2019), 4878–4885. <https://doi.org/10.1609/aaai.v33i01.33014878>
- [36] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. 2019. Dyrep: Learning representations over dynamic graphs. In *International conference on learning representations*.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [38] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2017. Graph Attention Networks. *6th International Conference on Learning Representations* (2017).
- [39] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. 2021. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 international conference on management of data*, 2628–2638.
- [40] Zhihao Wen and Yuan Fang. 2022. TREND: TempoRal Event and Node Dynamics for Graph Representation Learning. In *Proceedings of the ACM Web Conference 2022* (Virtual Event, Lyon, France) (WWW '22). Association for Computing Machinery, New York, NY, USA, 1159–1169. <https://doi.org/10.1145/3485447.3512164>
- [41] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying Graph Convolutional Networks. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 6861–6871. <https://proceedings.mlr.press/v97/wu19e.html>
- [42] Yongji Wu, Defu Lian, Yiheng Xu, Le Wu, and Enhong Chen. 2020. Graph Convolutional Networks with Markov Random Field Reasoning for Social Spammer Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* (Jun 2020), 1054–1061. <https://doi.org/10.1609/aaai.v34i01.5455>
- [43] Menglin Yang, Min Zhou, Marcus Kalander, Zengfeng Huang, and Irwin King. 2021. Discrete-time temporal network embedding via implicit hierarchical learning in hyperbolic space. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 1975–1985.

- [44] Jiaxuan You, Tianyu Du, and Jure Leskovec. 2022. ROLAND: Graph Learning Framework for Dynamic Graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) (KDD '22). Association for Computing Machinery, New York, NY, USA, 2358–2366. <https://doi.org/10.1145/3534678.3539300>
- [45] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*. 3634–3640.
- [46] Kaike Zhang, Qi Cao, Gaolin Fang, Bingbing Xu, Hongjian Zou, Huawei Shen, and Xueqi Cheng. 2023. DyTed: Disentangled Representation Learning for Discrete-time Dynamic Graph. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3309–3320.
- [47] Yanping Zheng, Lu Yi, and Zhewei Wei. 2024. A survey of dynamic graph neural networks. *Frontiers of Computer Science* 19, 6 (12 Dec 2024), 196323. <https://doi.org/10.1007/s11704-024-3853-2>
- [48] Yifan Zhu, Fangpeng Cong, Dan Zhang, Wenwen Gong, Qika Lin, Wenzheng Feng, Yuxiao Dong, and Jie Tang. 2023. WinGNN: Dynamic Graph Neural Networks with Random Gradient Aggregation Window. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) (KDD '23). Association for Computing Machinery, New York, NY, USA, 3650–3662. <https://doi.org/10.1145/3580305.3599551>
- [49] Yuan Zuo, Guannan Liu, Hao Lin, Jia Guo, Xiaoqian Hu, and Junjie Wu. 2018. Embedding Temporal Network via Neighborhood Formation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (KDD '18). Association for Computing Machinery, New York, NY, USA, 2857–2866. <https://doi.org/10.1145/3219819.3220054>
- [50] Yuan Zuo, Guannan Liu, Hao Lin, Jia Guo, Xiaoqian Hu, and Junjie Wu. 2018. Embedding temporal network via neighborhood formation. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2857–2866.

A Research Methods

A.1 Details of Equation 8

In this subsection, we provide details of Equation 8, proving $tr(\mathbf{F}^T \mathbf{L} \mathbf{F}) = \sum_{(i,j) \in \mathcal{E}} C_{ij} \|\mathbf{F}_i - \mathbf{F}_j\|_2^2$, while $\mathbf{L} = \text{diag}(\sum_j C_{ij}) - \mathbf{C}$. For the sake of convenience in symbolic representation, let us assume that $\mathbf{F} \in \mathbb{R}^{n \times 1}$ with elements denoted as $f_1, f_2, \dots, f_n$, and the process of proving in the high-dimensional case follows the same steps. By using the rule of matrix trace calculation, the left term can be changed into

$$\begin{aligned} tr(\mathbf{F}^T \mathbf{L} \mathbf{F}) &= tr(\mathbf{F} \mathbf{F}^T \mathbf{L}) \\ &= tr(\mathbf{F} \mathbf{F}^T \text{diag}(\sum_j C_{ij})) - tr(\mathbf{F} \mathbf{F}^T \mathbf{C}). \end{aligned}$$

Where the elements of $\mathbf{F} \mathbf{F}^T$ is

$$\begin{bmatrix} f_1 f_1 & f_1 f_2 & \cdots & f_1 f_n \\ f_2 f_1 & f_2 f_2 & \cdots & f_2 f_n \\ \vdots & \vdots & \ddots & \vdots \\ f_n f_1 & f_n f_2 & \cdots & f_n f_n \end{bmatrix}.$$

Since $\text{diag}(\sum_j C_{ij})$ is a diagonal matrix

$$\begin{bmatrix} \sum_j C_{1j} & & & \\ & \ddots & & \\ & & \ddots & \\ & & & \sum_j C_{nj} \end{bmatrix}.$$

Therefore we have

$$tr(\mathbf{F} \mathbf{F}^T \text{diag}(\sum_j C_{ij})) = \sum_i \sum_j C_{ij} f_i^2.$$

Since the trace operation only sums elements in diagonal, we have

$$tr(\mathbf{F} \mathbf{F}^T \mathbf{C}) = \sum_i \sum_j C_{ij} f_i f_j.$$

Based on the above results, equation $tr(\mathbf{F} \mathbf{F}^T \text{diag}(\sum_j C_{ij})) - tr(\mathbf{F} \mathbf{F}^T \mathbf{C})$ can be transformed into

$$\begin{aligned} &= \sum_i \sum_j C_{ij} f_i^2 - \sum_i \sum_j C_{ij} f_i f_j \\ &= \frac{1}{2} \left(\sum_i \sum_j C_{ij} f_i^2 - 2 \sum_i \sum_j C_{ij} f_i f_j + \sum_i \sum_j C_{ij} f_j^2 \right) \\ &= \frac{1}{2} \sum_i \sum_j C_{ij} (f_i - f_j)^2 \\ &= \sum_{(i,j) \in \mathcal{E}} C_{ij} \|\mathbf{F}_i - \mathbf{F}_j\|_2^2, \end{aligned}$$

which completes the proof.

A.2 Complexity Analysis

In this subsection, we provide the detailed descriptions for time and space analysis in Table 1.

Time complexity. According to the results presented ClusterGCN [3], the time complexity of message-passing based GNN is given by $O(lf + lnf^2)$. The DySAT involves obtaining node embeddings for t snapshots within a given time window, which requires a time complexity of $O(tlef + tlnf^2)$. Following this, the self-attention is utilized to encode temporal information for n nodes in t snapshots. The time complexity of the self-attention mechanism is $O(nt^2 f)$, while the feature transformation complexity is $O(nf^2)$. Consequently, the time complexity of DySAT can be expressed as $O(tlef + tlnf^2 + nt^2 f)$. Considering usually $lf > t$, then it can be written as $O(tlef + tlnf^2)$. In the case of EvolveGCN, RNN is used to update the parameters at each GNN step in the time window, and RNN performs feature transformation for all nodes in each snapshot, resulting in a time complexity of $O(nf^2)$. Thus, the total time complexity of EvolveGCN is $O(tlef + tlnf^2)$. In the Roland algorithm, the time window t is set to 1, and no time encoder is utilized, leading to a complexity of $O(lf + lnf^2)$ for a single window. SFDyG merges t snapshots in the time window into a temporal graph, with the number of edges being te . Therefore, for the full-batch training of SFDyG, denoted as SFDyG-F, the time complexity is $O(tlef + lnf^2)$. For the mini-batch training version of SFDyG denoted as SFDyG-M, each edge requires feature aggregation from $O(2d^l)$ neighbors, resulting in a time complexity of $O(2ed^l f^2)$.

Memory complexity. Similarly, Based on the conclusions from the ClusterGCN [3], the message passing based GNN has a space complexity of $O(lf + lf^2)$. Consequently, the space complexity of the GNN component in DySAT is $O(tlnf + tlf^2)$. The self-attention mechanism requires storing the attention matrix with $O(t^2)$, outputs $O(tf)$, and feature transformation parameters $O(f^2)$. Therefore, the space complexity of the self-attention component becomes $O(nt^2 + ntf + f^2)$. Typically, with $lf > t$, leading to an overall space complexity of $O(tlnf + tlf^2)$ for DySAT. The RNN part of EvolveGCN needs to store all intermediate results as parameters for GNN, resulting in a space complexity of $O(tnfl + f^2)$. Consequently, the combined space complexity becomes $O(tlnf + tlf^2)$. As Roland has a window size of 1, its space complexity is the same as that of a single GNN, which is $O(lf + lf^2)$. In the case of the full-batch SFDyG-F, the input edge number is te , resulting in a space complexity of $O(tlef + lf^2)$. For the mini-batch trained SFDyG-F,

a mini-batch can have bd^L edges, resulting in a space complexity of $O(2bd^L f + lf^2)$.

B Experiment Details

B.1 Description of Datasets

In our experiments, we utilize a combination of synthetic and publicly available benchmark.

Stochastic Block Model.¹ (SBM for short) SBM is a widely employed random graph model utilized for simulating community structures and evolutions. The data utilized in this study was obtained from the GitHub repository of EvolveGCN [30].

Bitcoin OTC.² (OTC for short) OTC is a who-trusts-whom network among bitcoin users who engage in trading activities on the platform <http://www.bitcoin-otc.com>. The data set may be used for predicting the polarity of each rating and forecasting whether a user will rate another one in the next time step.

Bitcoin Alpha.³ (Alpha for short) Alpha is created in the same manner as OTC, except that the users and ratings come from a different trading platform, <http://www.btc-alpha.com>.

UC Irvine messages.⁴ (UCI for short) UCI is an online community of students from the University of California, Irvine, wherein the links of this social network indicate sent messages between users. Link prediction is a standard task for this data set.

Autonomous systems.⁵ (AS for short) Autonomous Systems (AS) constitute a communication network of routers that exchange traffic flows with their peers. This dataset can be utilized for predicting future message exchanges.

Reddit-Title and Reddit-Body.⁶ The network of hyperlinks between subreddits is derived from hyperlinks found in posts. These hyperlinks can appear in either post titles or bodies, resulting in two distinct datasets.

Stack Overflow.⁷ (SO for short) The dataset presents interactions within the Stack Overflow platform. Nodes in the dataset represent users, while directed edges indicate the flow of answer activity between users.

B.2 Description of Baselines

DySAT [34] computes node representations through joint self-attention along the two dimensions of the structural neighborhood and temporal dynamics.

EvolveGCN [30]: adapts the GCN to compute node representations, and captures the dynamism of the graph sequence by using an RNN to evolve the GCN parameters.

ROLAND [44]: views the node representations at different GNN layers as hierarchical node states and recurrently updates them. We present results of the moving average variant of Roland, which can be trained in our GPU environment for the StackOverflow dataset.

VGRNN [12]: a hierarchical variational model that introduces additional latent random variables to jointly model the hidden states of a graph recurrent neural network (GRNN).

HTGN [43] maps the dynamic graph into hyperbolic space, and incorporates hyperbolic graph neural network and hyperbolic gated recurrent neural network to obtain representations.

WinGNN [48]: is a GNN model that employs a meta-learning strategy and introduces a novel random gradient aggregation mechanism.

GraphMixer [6]: simplifies temporal graph learning by utilizing MLP-based link and node encoders, achieving high performance in link prediction tasks through faster convergence and improved generalization.

M2DNE [27]: is a novel approach for temporal network embedding by effectively capturing both micro- and macro-dynamics through a temporal attention point process and a general dynamics equation parameterized with network embeddings.

GHP [35]: integrates Hawkes processes with a graph convolutional recurrent neural network to efficiently model correlated temporal sequences with improved prediction accuracy.

B.3 Evaluation Metrics

The MRR score is the average of the reciprocal ranks of the positive samples within the negative samples, formally,

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank(p_i)}.$$

B.4 Experiment Setup

B.4.1 Running Environment. We perform our comparisons on an Ubuntu 20.04.4 LTS server with an Intel Xeon 32-Core Processor, 200 GB RAM, and an NVIDIA A100-SXM4-80GB Tensor Core GPU. SFDyG is implemented with Python 3.11.4 with PyTorch 2.3.1 and torch_geometric 2.4.0 framework.

B.4.2 Hyper-parameter Settings. In all experiments, unless otherwise specified, we ensured consistency by utilizing a standardized set of hyperparameters, with dummy node features (a vector of all ones) to avoid over-fitting, one negative sample for training, and 100 negative samples for testing. Negative samples for testing were generated in advance to maintain reproducibility. The training window size was set to 10, comprising 9 input snapshots and one target snapshot. The default dropout rate was 0.1, the learning rate was 0.001, the hidden layer size was 64 and the patience of early stopping was 20 epochs without MRR increase on the validation dataset. For the mini-batch experiments, *LinkNeighborLoader* in PyG [9] was used to construct the mini-batches. Additionally, the Adam [21] was used as the optimizer for gradient descent and the Cosine Annealing learning rate scheduler [26] was used to accelerate training.

B.4.3 Source Code. The source code is available at <https://github.com/oncemoe/hawkesGNN>

¹<https://github.com/IBM/EvolveGCN>

²<http://snap.stanford.edu/data/soc-sign-bitcoin-otc.html>

³<http://snap.stanford.edu/data/soc-sign-bitcoin-alpha.html>

⁴<http://konect.uni-koblenz.de/networks/opsahl-ucsocial>

⁵<http://snap.stanford.edu/data/as-733.html>

⁶<https://snap.stanford.edu/data/soc-RedditHyperlinks.html>

⁷<https://snap.stanford.edu/data/sx-stackoverflow.html>